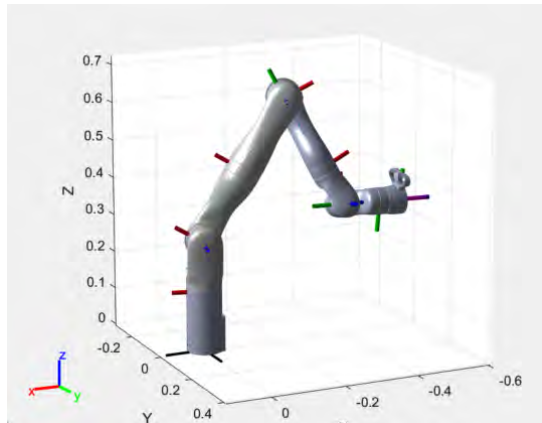




Industrial Robotics (Laboratory)

Eric Kaigom

Labs Industrial Robotics (SS 23, for course purposes only).

Contents

1	Introduction	3
1.0.1	Safety-related instructions	5
1.0.2	Safety-related functions in the laboratory room	5
2	First Assignment - 3D Modeling: Frame Manipulation	7
2.1	Objectives	7
2.2	Preliminaries	7
2.3	Drawing frames	9
2.4	Spatial transformation between frames	9
2.5	Transformation of point coordinates from a frame to another	10
3	Second Assignment - Kinematics: Forward Transformation and Manipulability Monitoring of a 7DoF Physical Robot	11
3.1	Objectives	11
3.2	Deriving and computing the forward kinematics of a physical robot	12
3.3	Monitoring the manipulability of a physical manipulator by using its digital twin	13
4	Third Assignment - Dynamics and Sensors: Grasp-and-Place Workpieces and Energy Consumption Monitoring	15
4.1	Objectives	15
4.2	Preliminaries	15
4.3	Planning and executing collision-free motion of the physical robot for pick-and-place-maneuvers	17
4.4	Calculating the energy consumption of the physical robot	18

1 Introduction

Throughout the first assignment, you will experiment how **frames** spatially transform. The practical relevance of this assignment is closely related to task planning and execution in (industrial) robotized applications. In this context, it is often useful to express the position and orientation (i.e., pose) of the end-effector or a workpiece in an intuitive, advantageous, or simply convenient frame. For instance, the pose of a gripper attached to the last link of a robot manipulator is typically described w.r.t a frame fixed on the same link (see fig. 1.1). Such a transformation is generally constant unless the frame is fixed on articulated links (e.g., both fingers in fig. 1.1) of the gripper. Varying transformations occur when

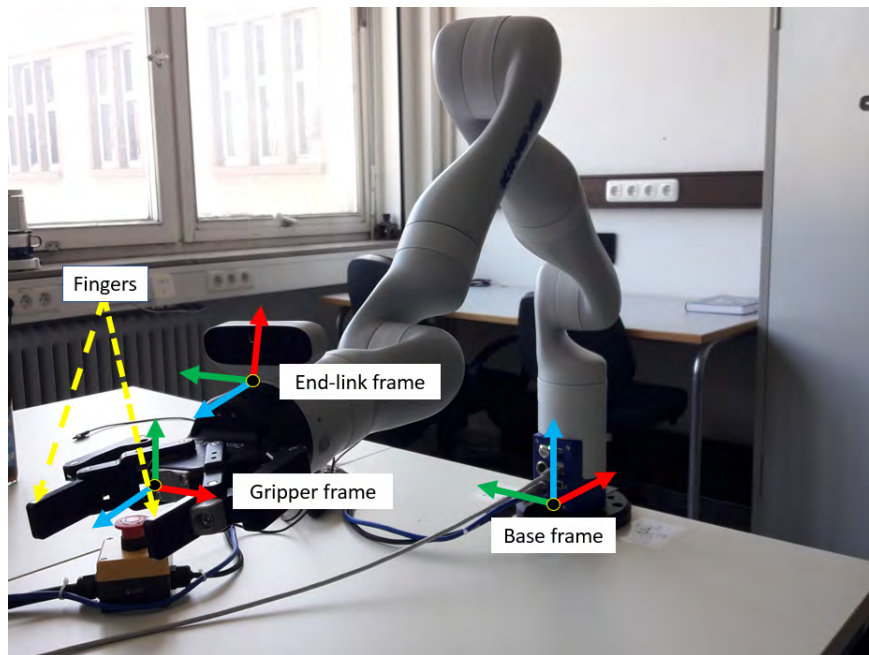


Figure 1.1: An example of spatial frame placement. Whereas the end-link and gripper frames have a constant relative pose, the pose of each frame varies when perceived from the base frame. The variation depends upon the current posture (i.e., joint positions) of the manipulator.

they depend upon, for instance, the joints of the robot arm. In this respect, the mapping from the end-effector to a payload frame before grasping or the base frame (see fig. 1.1) is an example. Therefore, the first assignment gives you the opportunity to gain insights into how frames transform points in the Cartesian space through explicit practices.

In the second assignment, you will derive *in detail* the basic and overall transformations that govern the **forward kinematics (FK)** of the **physical robot manipulator** depicted in fig. 1.1. By doing so, you acquire an understanding of and practical experience with the design and functioning of **physical robot arms**. This is going to happen in a hybrid approach. Specifically, the derivation and construction of the FK is carried out in Matlab. In this software environment, you can flexibly design, test, and troubleshoot your implementations whenever and wherever (i.e., even in the train or at home) you want without any physical constraints associated with the targeted and digitally twinned physical robot hardware. Nevertheless, obtained simulation results will be compared with the FK used by the control unit of the corresponding physical robot. Hence, you will develop competencies in setting up and employing a **Software Development Kit (SDK)** to collect and scrutinize as well as assess key indexes reflected by data outputted by the physical robot in practice. This aspect is an important component of operating modern robotized systems as a pervasive and intelligent entity in future smart factories.

Equipped with the aforementioned background and insight, you will plan and implement the motion of a physical robot to solve a **grasp-and-place task** in the final third appointment. Such tasks are preponderant in the manufacturing and assembly industry. Therefore, you will be exposed to planning challenges, including **collision** and **singularity avoidance**. The **manipulability** of the robot will be monitored (i.e., visualized) and discussed during its motion. Associated **actuation data** that characterize the **dynamics** of the physical robot will be collected by reading built-in **sensors** via the robot SDK in Matlab to compute, visualize, and discuss the behavior of its energy consumption.

At the end of the assignments, you will have nurtured your awareness of the **safety, human-centered, technical, and economical** implications of robotics in the industrial realm. This will be done in terms of e.g., hazards (e.g., collisions) management, usability, and augmentation of human (e.g., visualized manipulability, decision support), proficiency and performance in real-time, as well as potential costs. An advantage of these industrial values considered throughout the three assignments is to experiment how combining emerging digital (e.g., data-

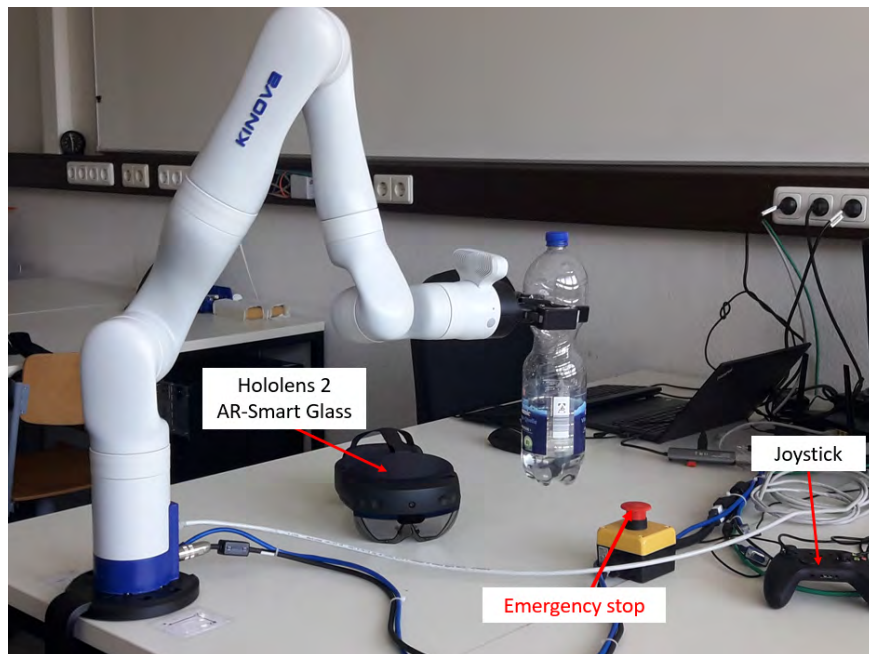


Figure 1.2: Emergency stop as well as immersive and control devices.

driven and immersive) technologies and hardware understanding is instrumental in unleashing the potential of robots and reach a competitive edge in robotized applications. Interfaces toward this end (see, e.g., fig. 1.2) are available in our lab and can be leveraged to propel your creative ideas and implementations to the next frontiers upon request!

1.0.1 Safety-related instructions

Operating a robot is always prone to safety hazards. Before the labs and throughout lectures, instructions will be given about safety-related requirements in the lab. You are invited to carefully follow these instructions throughout the labs. Be advised that any misbehavior will be taken very seriously and might lead you to disqualification from the labs. In fact, *your safety is our top priority!*

1.0.2 Safety-related functions in the laboratory room

We follow the recommendations of ISO 10218-2 and ISO/TS 15066 to leverage the following functions:

- **Emergency stop:** Its **manual activation** is shown in fig. 1.2. Please, **press**



Figure 1.3: Red Emergency Stop button.

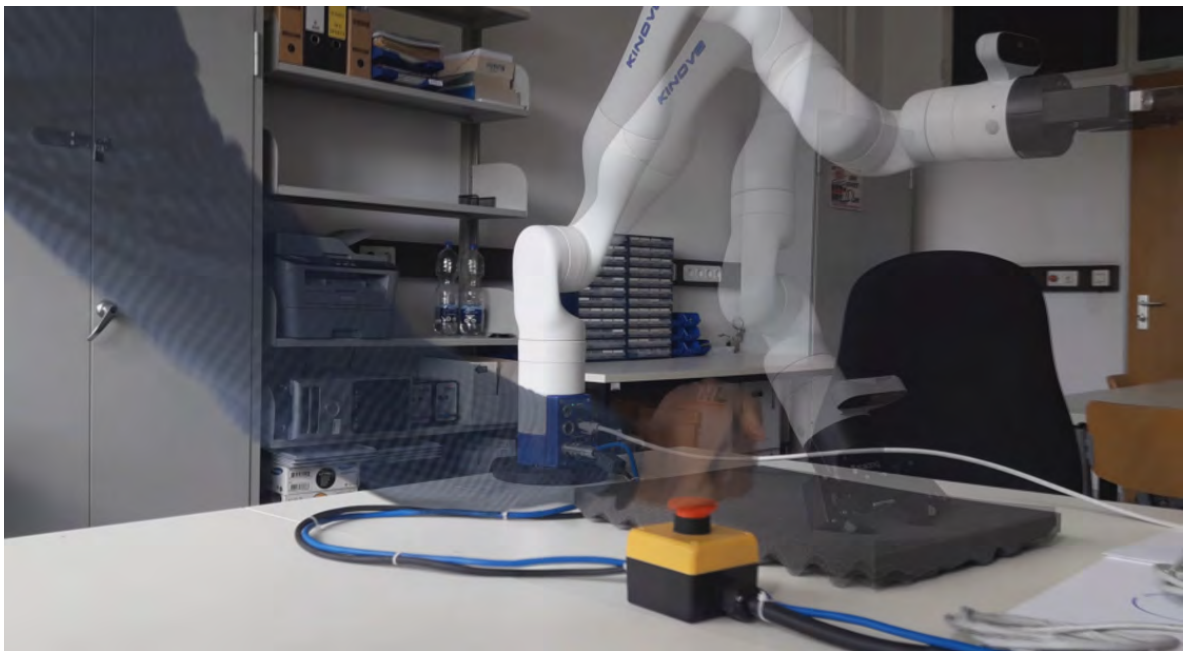


Figure 1.4: Robot behavior after pressing the red Emergency Stop button.

the **red button** in any emergency situation! The robot will stop moving and fall. However, the descent is rather slow. Once the power supply is cut, the control of the robot will be prevented (as shown in fig. 1.4) until this state is cleared.

- **Protective stop:** The robot stops its motion whenever an entity (e.g., a human, an object, etc.) is located within a predefined critical safety zone. In the case that the entity is located in an intermediate or gray safety zone outside the critical one, the robot velocity will be reduced to limit the robot momentum.

Please, stay *outside* the safety zone!

2 First Assignment - 3D Modeling: Frame Manipulation

2.1 Objectives

In this assignment, you will

1. draw targeted 3D frames in Matlab.
2. calculate homogeneous transformations between these 3D frames.
3. express coordinates of points in different frames.

2.2 Preliminaries

Open Matlab and familiarize yourself with the employment of the function

$$\text{plot3}(x,y,z) \quad (2.1)$$

by entering

$$\text{help plot3}. \quad (2.2)$$

Use this function to draw in **red**, **green**, and **blue** the three 3D vectors O_0A_0 along positive x -, O_0B_0 along positive y -, and O_0C_0 along positive z -coordinates of an orthonormal coordinate system. The three vectors have the common origin

$$O_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \quad (2.3)$$

and respective endpoint

$$A_0 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad (2.4)$$

$$B_0 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad (2.5)$$

and

$$C_0 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}. \quad (2.6)$$

The result is the world frame $\mathcal{F}_0(O_0, O_0A_0, O_0B_0, O_0C_0)$.

Now, consider the following frames

$$\mathcal{F}_1(O_1, O_1A_1, O_1B_1, O_1C_1), \quad (2.7)$$

$$\mathcal{F}_2(O_2, O_2A_2, O_2B_2, O_2C_2), \quad (2.8)$$

and

$$\mathcal{F}_3(O_3, O_3A_3, O_3B_3, O_3C_3). \quad (2.9)$$

Herein,

- $\mathcal{F}_1$ is the rotation by $-89^\circ, -175^\circ$ and 90° about O_0A_0, O_0B_0 , and O_0C_0 of a frame that initially matches with $\mathcal{F}_0$ followed by the translation

$$O_0O_1 = \begin{bmatrix} -0.78 \\ 0.15 \\ 0.21 \end{bmatrix} \quad (2.10)$$

- $\mathcal{F}_2$ is the rotation by -89° , -175° and 90° about 0_0A_0 , 0_0B_0 , and 0_0C_0 of a frame that initially matches with $\mathcal{F}_0$ followed by the translation

$$O_0O_2 = \begin{bmatrix} -0.78 \\ -0.05 \\ 0.21 \end{bmatrix} \quad (2.11)$$

- $\mathcal{F}_3$ is the rotation by 91° , -5° and 10° about 0_0A_0 , 0_0B_0 , and 0_0C_0 of a frame that initially matches with $\mathcal{F}_0$ followed by the translation

$$O_0O_3 = \begin{bmatrix} -0.55 \\ -0.2 \\ 0.21 \end{bmatrix}. \quad (2.12)$$

2.3 Drawing frames

Implement a Matlab-function

$$\textit{drawFrame}(O, R) \quad (2.13)$$

that receives a 3D point O (expressed in $\mathcal{F}_0$) and a rotation matrix R as origin (i.e., Cartesian position) and orientation of a targeted frame $\mathcal{F}_T$ relative to $\mathcal{F}_0$. The function *drawFrame* plots $\mathcal{F}_T$ in Matlab.

Use *drawFrame*(O, R) to draw $\mathcal{F}_1$, $\mathcal{F}_2$, and $\mathcal{F}_3$ in a Matlab-script!

The following functions from the *robotics toolbox* of Matlab might be helpful toward this end:

- *rotm2tform*
- *trvec2tform*

2.4 Spatial transformation between frames

Compute the homogeneous transformations:

- T_{01} from $\mathcal{F}_0$ to $\mathcal{F}_1$

- T_{21} from $\mathcal{F}_2$ to $\mathcal{F}_1$
- T_{32} from $\mathcal{F}_3$ to $\mathcal{F}_2$
- T_{20} from $\mathcal{F}_2$ to $\mathcal{F}_0$

2.5 Transformation of point coordinates from a frame to another

Consider the point $P_3 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ in $\mathcal{F}_3$ and $P_2 = \begin{bmatrix} -1 \\ 0 \\ -1 \end{bmatrix}$ in $\mathcal{F}_2$. Compute the coordinates of P_3 and P_2 in $\mathcal{F}_1$ and $\mathcal{F}_0$ in Matlab.

3 Second Assignment - Kinematics: Forward Transformation and Manipulability Monitoring of a 7DoF Physical Robot

3.1 Objectives

In this assignment, you will

1. derive the forward kinematics of a physical robot from a given placement of frames associated with its kinematic chain in Matlab.
2. compare this forward kinematics with the kinematics employed by the real controller of the physical robot.
3. provide the direction of the highest Cartesian motion capability as a vector having its initial point at the end-effector of a simulated robot.

Hint: Commands, functions, instructions, and data structures of the Matlab-based SDK/API that helps create and manipulate the simulated as well as connect to and operate the physical robot can be found in [Mat23a, Kin23].

Before sending pre-computed desired trajectories to the robot hardware, please read the following webpage [Mat23b] and strictly follow mentioned recommendations! Make sure that none is in the critical safety zone of the robot at any point of time (even when the robot is at rest!).

3.2 Deriving and computing the forward kinematics of a physical robot

The placement of frames on the kinematic chain of the physical robot in fig. 3.1a

is depicted in fig. 3.1b. Each joint locally rotates around the z-axis. Let $q = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ q_5 \\ q_6 \\ q_7 \end{bmatrix}$

denote the vector of current joint positions of the robot. Data about the geometry between the frames are obtained from fig. 3.1b.

1. Derive the homogeneous transformation $T_{01}(q_1)$ between the 0-th and 1-rst frame as a function of rotations and translations that depend upon q_1 and the geometry between both frames.
2. Derive the homogeneous transformation $T_{78}(q_7)$ between the 7-th and 8-th frame (of the interface module in fig. 3.1b) as a function of rotations and translations that depend upon q_7 and the geometry between both frames.
3. Derive (i.e., *with a well-founded justification of involved basic transformations*) in a similar way the homogeneous transformations $T_{12}, T_{23}, T_{34}, T_{45}, T_{56}, T_{67}$.
4. Implement a Matlab script which is fed with q as parameter and returns the full forward kinematics $T_{08}(q)$ of the robot as a matrix! Observe that the tool is located at the *local* position vector $v = [0 \ 0 \ 0.12]^T$ at the end-effector.
5. **Safely** drive the simulated robot (by using $T_{08}(q)$) and physical robot (by commanding postures) to the consecutive joint configurations (in degrees):

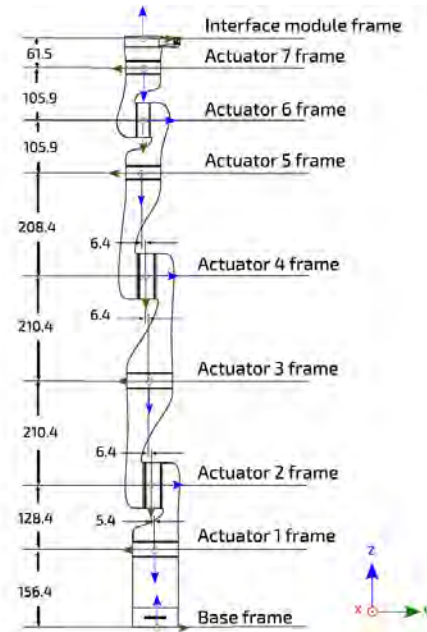
$$q_{\mathcal{F}_1} = \begin{bmatrix} 16.39 \\ 299.74 \\ 5.1 \\ 268.15 \\ 22.51 \\ 63.48 \\ 71.2 \end{bmatrix}, q_{\mathcal{F}_2} = \begin{bmatrix} 351.73 \\ 300.69 \\ 8.11 \\ 263.95 \\ 356.98 \\ 63 \\ 79.94 \end{bmatrix}, \text{ and } q_{\mathcal{F}_3} = \begin{bmatrix} 1.18 \\ 291.32 \\ 18.47 \\ 290.91 \\ 94.36 \\ 112.93 \\ 46 \end{bmatrix}.$$

A prior evaluation of robot motions in Matlab before the transfer to the hardware is highly recommended! How did you ensure safety?

3.3 Monitoring the manipulability of a physical manipulator by using its digital twin



(a) Physical robot.



(b) Frame placement (@kinova.com).

Figure 3.1: The physical robot. A higher resolution of fig. 3.1b can be found in the same folder as this document.

6. Collect the frames of the end-effector (Interface Module in fig. 3.1b) at $q = q_{\mathcal{F}_1}$, $q = q_{\mathcal{F}_2}$, and $q = q_{\mathcal{F}_3}$ for the simulated and physical robots. How do the simulated and real frames line up? Did you successfully capture the forward kinematics of the **physical robot**?

3.3 Monitoring the manipulability of a physical manipulator by using its digital twin

1. Take advantage of the concept of manipulability to identify the translational direction v_m of the highest motion capability of the end-effector at any time of the robot simulation while moving to $q = q_{\mathcal{F}_1}$, $q = q_{\mathcal{F}_2}$, and $q = q_{\mathcal{F}_3}$ as defined in section 3.2.
2. Plot the vector v_m in magenta with its initial point at the origin of the frame of the end-effector at any instant of time while the robot is moving to $q = q_{\mathcal{F}_1}$, $q = q_{\mathcal{F}_2}$, and $q = q_{\mathcal{F}_3}$ as defined in section 3.2.
3. Interpret the varying behavior of v_m for different configurations of the ma-

3.3 Monitoring the manipulability of a physical manipulator by using its digital twin

nipulator. Consider and justify in particular the behavior of v_m at the initial and final configuration of the simulated manipulator.

4 Third Assignment - Dynamics and Sensors: Grasp-and-Place Workpieces and Energy Consumption Monitoring

4.1 Objectives

In this assignment, you will

1. plan collision-free point-to-point (in the joint) and continuous path (in the Cartesian Space) motions to reach frames in the task space of a physical robot. This will be completed in Matlab first.
2. transfer those motions to the physical robot to grasp and modify the pose of two bottles as workpieces in the manufacturing industry.
3. collect dynamics-related data, compute, and plot the energy consumption of the robot.

4.2 Preliminaries

Two similar bottles B_1 and B_2 are initially located at the grasp frame $\mathcal{F}_1$ and $\mathcal{F}_3$ specified in section 2.2. $\mathcal{F}_2$, which is also defined in section 2.2, is an intermediate frame.

The goal of the following tasks is to safely operate the robot to grasp and modify the location of B_1 and B_2 . Specifically, B_1 and B_2 are located at the frame $\mathcal{F}_3$ and $\mathcal{F}_2$ at the end of the maneuver. These displacements are illustrated in fig. 4.1.

The gripper can *hold* a bottle at any collision-free joint configuration while the

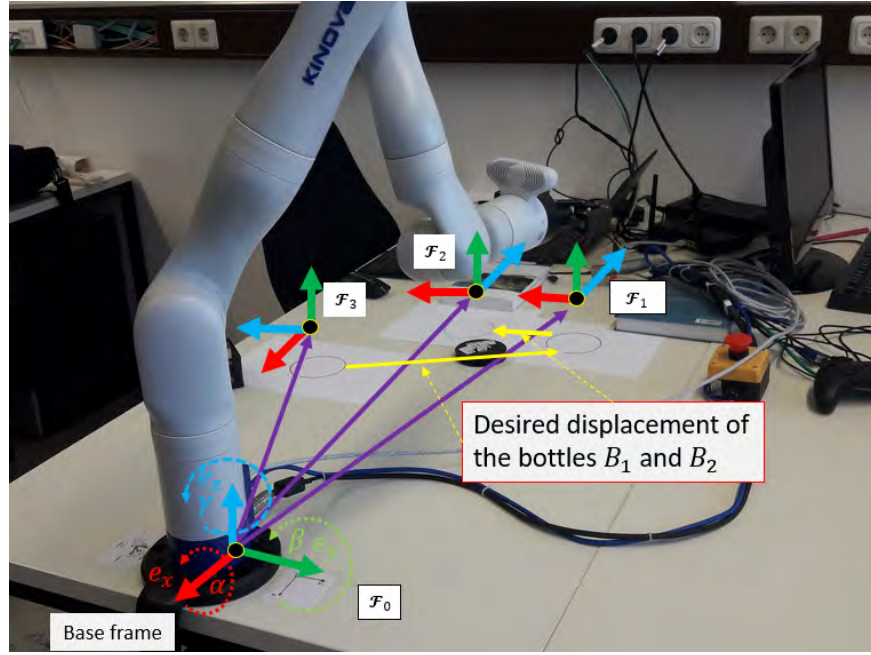


Figure 4.1: Frame and task objectives (displacements are depicted by the yellow arrows).

robot is in motion or not. However, the gripper is *not* allowed to *release* the bottle at a frame that differs from $\mathcal{F}_1$, $\mathcal{F}_2$, and $\mathcal{F}_3$.

4.3 Planning and executing collision-free motion of the physical robot for pick-and-place-maneuvers

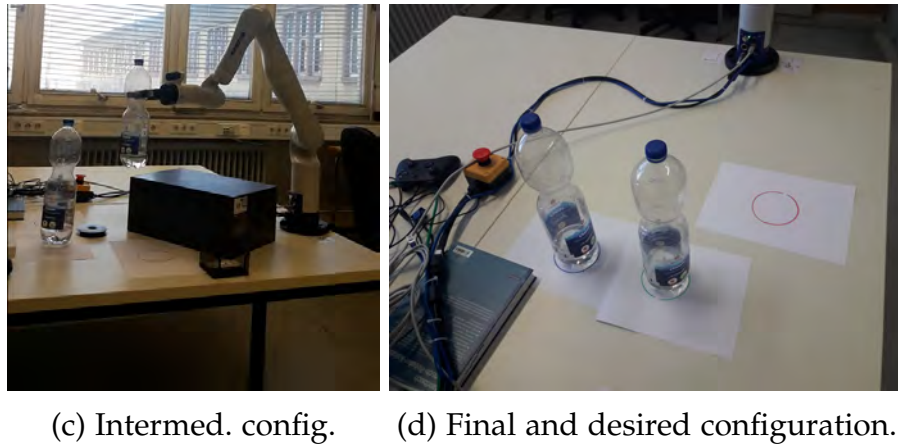
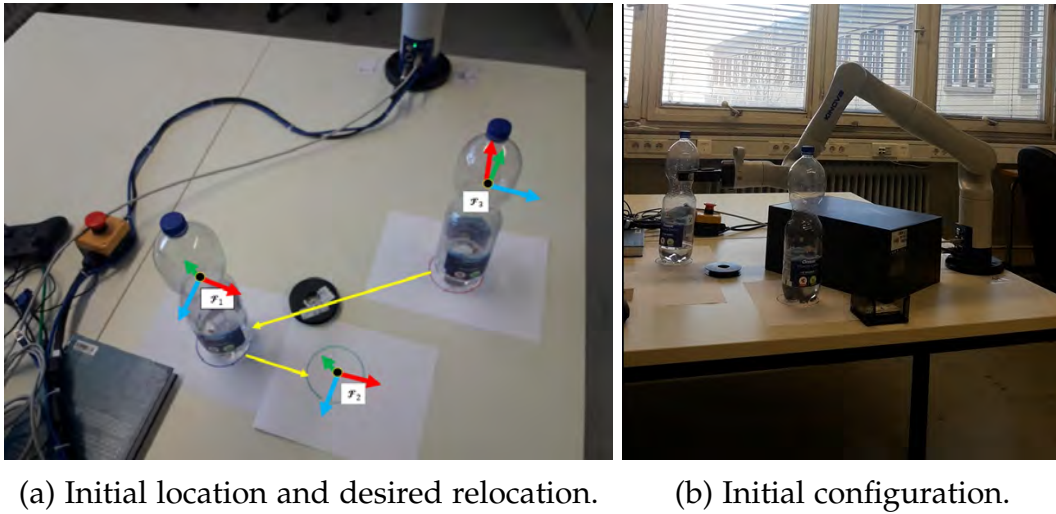


Figure 4.2: Experimental setup, task objective, initial and final configuration of targeted workpieces.

4.3 Planning and executing collision-free motion of the physical robot for pick-and-place-maneuvers

Design, implement, and execute a sequence of collision-free motion commands through which the physical robot uses its gripper to reach the desired location of B_1 and B_2 on the table (see fig. 4.2).

4.4 Calculating the energy consumption of the physical robot

Collect energy-relevant sensor data while switching the bottles. Use gathered data to calculate and plot the energy consumption of the robot. Interpret the variation of the energy consumption of the physical robot in the different motion phase.

Bibliography

- [Kin23] Kinova, 1Matlab: *Open and close the gripper, send cartesian pose command*, 2023 [Online; accessed 11-Apr-2023]. <https://de.mathworks.com/help/supportpkg/robotmanipulator/ug/connect-kinova-manipulate-arm.html#ConnectKinovaGen3RobotManipulateArmMATLABExample-8>.
- [Mat23a] Matlab, 2Kinova: *connect to robot, visualize actual movement, send robot to desired configuration, send precomputed trajectory*, 2023 [Online; accessed 11-Apr-2023]. <https://de.mathworks.com/help/supportpkg/robotmanipulator/ug/track-trajectory-inverse-kinematics.html#TrackPreComputedTrajectoryKinovaGen3RobotExample-12>.
- [Mat23b] Matlab, 3Kinova: *Precomputed joint trajectories*, 2023 [Online; accessed 11-Apr-2023]. https://github.com/Kinovarobotics/matlab_kortex/blob/master/simplified_api/documentation/precomputed_joint_trajectories.md.